

# Capítulo 2

## Arquitectura Full Stack y ciclo de desarrollo

Desarrollo de Software IX · Código 1493 · Módulo II

### Propósito del capítulo

Comprender cómo se organiza una solución Full Stack, qué responsabilidades tiene cada capa y cómo se conectan decisiones técnicas con el ciclo de desarrollo de una aplicación web profesional.

# Tabla de contenido

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>1. Panorama del capítulo</b>                                | <b>3</b>  |
| <b>2. Evolución histórica de las arquitecturas web</b>         | <b>3</b>  |
| <b>3. Teoría de capas: acoplamiento y cohesión</b>             | <b>5</b>  |
| <b>4. Características del desarrollo Full Stack</b>            | <b>5</b>  |
| <b>5. Tipos de desarrollo Full Stack</b>                       | <b>6</b>  |
| <b>6. Frameworks frontend</b>                                  | <b>6</b>  |
| <b>7. Frameworks backend</b>                                   | <b>7</b>  |
| <b>8. Integración frontend-backend</b>                         | <b>7</b>  |
| <b>9. Stacks tecnológicos más utilizados</b>                   | <b>8</b>  |
| <b>10. El rol profesional del desarrollador Full Stack</b>     | <b>9</b>  |
| <b>11. El ciclo de desarrollo de una aplicación Full Stack</b> | <b>10</b> |
| <b>12. Buenas prácticas</b>                                    | <b>11</b> |
| <b>13. Caso guía: seguimiento de solicitudes académicas</b>    | <b>11</b> |
| <b>14. Actividades sugeridas</b>                               | <b>12</b> |
| <b>15. Rúbrica breve</b>                                       | <b>12</b> |
| <b>16. Cierre</b>                                              | <b>12</b> |

# 1. Panorama del capítulo

Este capítulo presenta la arquitectura Full Stack como una forma de organizar responsabilidades dentro de un sistema de software. Una aplicación completa no es solo una interfaz atractiva ni un servidor aislado: es la coordinación deliberada entre experiencia de usuario, servicios, reglas de negocio, persistencia, seguridad y despliegue. El estudiante encontrará aquí la historia que explica por qué existen las arquitecturas actuales, la teoría de capas que las sustenta, el panorama de frameworks y stacks que las materializan, y el ciclo de desarrollo que las convierte en producto.

## Idea central

Full Stack significa poder razonar sobre el sistema completo. No exige ser experto absoluto en todas las capas, pero sí comprender cómo una decisión en una capa afecta a las demás: un cambio en el modelo de datos repercute en la API, y un cambio en la API repercute en cada pantalla que la consume.

| Capa          | Responsabilidad                                                                   | Preguntas de diseño                                                                     |
|---------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Frontend      | Presentar información, capturar interacción y consumir servicios.                 | ¿Qué necesita ver el usuario? ¿Qué estados debe entender? ¿Qué errores deben mostrarse? |
| Backend       | Exponer reglas de negocio, validar datos, proteger recursos e integrar servicios. | ¿Qué rutas existen? ¿Quién puede usarlas? ¿Qué se valida antes de guardar?              |
| Base de datos | Persistir entidades del dominio y permitir consultas consistentes.                | ¿Qué se guarda? ¿Qué relaciones existen? ¿Qué datos deben ser únicos?                   |
| Operación     | Configurar ambiente, variables, despliegue, logs y monitoreo básico.              | ¿Cómo se ejecuta? ¿Qué falla puede diagnosticarse? ¿Qué secretos deben protegerse?      |

## 2. Evolución histórica de las arquitecturas web

Las arquitecturas de software no surgieron de un manual: son respuestas acumuladas a problemas reales de escala, mantenimiento y trabajo en equipo. Entender esa evolución evita un error frecuente del desarrollador novato, que consiste en adoptar la arquitectura más reciente sin preguntarse qué problema resuelve. Cada etapa que se describe a continuación sigue vigente en la industria; lo que cambia es el contexto en el que cada una resulta apropiada.

### 2.1 El monolito: todo en un mismo lugar

En las primeras aplicaciones web, el servidor generaba el HTML completo de cada página, ejecutaba la lógica de negocio y consultaba la base de datos dentro de un solo programa desplegado como una unidad. A este estilo se le llama monolito. Su gran virtud es la simplicidad operativa: un solo repositorio, un solo despliegue, un solo lugar donde buscar errores. Su debilidad aparece con el crecimiento: cuando cientos de funcionalidades comparten el mismo código y la misma base de datos, cualquier cambio puede romper algo lejano, y los equipos terminan estorbándose entre sí. Conviene subrayar que un monolito bien modularizado sigue siendo una opción legítima y frecuente para equipos pequeños y productos en etapa temprana.

## 2.2 Cliente-servidor y la separación de responsabilidades

El modelo cliente-servidor formaliza una división que el monolito mantenía implícita: el cliente, usualmente el navegador, ejecuta la interfaz y envía solicitudes; el servidor recibe esas solicitudes, aplica reglas y devuelve respuestas. Esta separación permite que la interfaz evolucione sin mezclar reglas sensibles, y que el backend centralice datos, seguridad e integraciones. En la web moderna, el cliente se comunica con el servidor mediante el protocolo HTTP y recibe datos en formato JSON. El navegador nunca debería conectarse directamente a la base de datos ni conocer secretos de infraestructura: el backend funciona como frontera entre el usuario y los recursos críticos.

## 2.3 Las SPA: el cliente se vuelve aplicación

Hacia la década de 2010, la combinación de JavaScript más capaz y APIs HTTP bien definidas hizo posible que la interfaz dejara de ser páginas generadas por el servidor y se convirtiera en una aplicación que vive en el navegador. Las Single Page Applications (SPA) cargan una vez y luego actualizan la pantalla consumiendo servicios, sin recargar la página completa. Este giro tuvo dos consecuencias profundas: la experiencia de usuario ganó fluidez comparable a la de una aplicación de escritorio, y el backend pudo concentrarse en exponer datos y reglas a través de una API, sirviendo por igual a un navegador, a una aplicación móvil o a otro sistema. El costo fue una mayor complejidad en el cliente, que ahora administra estado, navegación y errores por su cuenta.

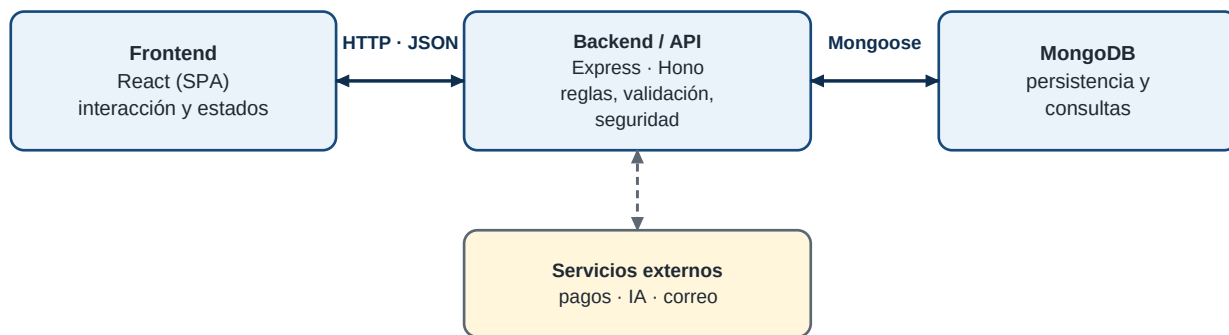
## 2.4 Microservicios: dividir el backend

Cuando organizaciones grandes alcanzaron los límites del monolito, surgió la arquitectura de microservicios: el backend se divide en servicios pequeños e independientes, cada uno con su propio despliegue y, con frecuencia, su propia base de datos. Los microservicios permiten que equipos distintos evolucionen partes distintas del sistema sin coordinarse en cada cambio, y que cada servicio escale según su demanda real. A cambio, introducen problemas que el monolito no tenía: comunicación por red entre servicios, consistencia de datos distribuida y una operación considerablemente más exigente. Para un equipo pequeño, adoptar microservicios prematuramente suele multiplicar la complejidad sin entregar beneficio.

### Lección de la historia

Ninguna arquitectura es superior en abstracto. El monolito optimiza la simplicidad; el modelo cliente-servidor optimiza la separación de responsabilidades; la SPA optimiza la experiencia de usuario; los microservicios optimizan la autonomía de equipos a gran escala. La pregunta profesional no es cuál está de moda, sino cuál se ajusta al tamaño del problema y del equipo.

| Etapas           | Fortaleza principal                     | Costo principal                         | Contexto típico                                 |
|------------------|-----------------------------------------|-----------------------------------------|-------------------------------------------------|
| Monolito         | Simplicidad de desarrollo y despliegue. | Acoplamiento creciente con el tamaño.   | Productos nuevos, equipos pequeños.             |
| Cliente-servidor | Frontera clara entre interfaz y reglas. | Coordinación de contratos entre partes. | Base de casi toda aplicación web actual.        |
| SPA + API        | Experiencia fluida; API reutilizable.   | Complejidad de estado en el cliente.    | Aplicaciones interactivas con varios clientes.  |
| Microservicios   | Autonomía de equipos y escalado fino.   | Operación distribuida y consistencia.   | Organizaciones grandes con dominios separables. |



el navegador NUNCA habla directo con la base ni con los secretos

### 3. Teoría de capas: acoplamiento y cohesión

Detrás de toda la evolución anterior operan dos conceptos clásicos de la ingeniería de software. La cohesión mide qué tan relacionadas están las responsabilidades que conviven dentro de un módulo: un módulo cohesivo hace una cosa bien definida. El acoplamiento mide cuánto depende un módulo de los detalles internos de otro: dos módulos fuertemente acoplados no pueden cambiar por separado. El diseño en capas persigue una meta simple de enunciar y difícil de sostener —alta cohesión dentro de cada capa y bajo acoplamiento entre capas— y la asegura con una regla de dependencia: cada capa solo conoce a la capa inmediatamente inferior a través de una interfaz, nunca sus detalles internos. El frontend conoce las rutas y los formatos de la API, pero no sabe qué base de datos hay detrás; el backend conoce el modelo de datos, pero no sabe qué componente de interfaz mostrará el resultado. Gracias a esta regla, es posible reemplazar la base de datos, rediseñar la interfaz o reescribir un servicio sin que el cambio se propague por todo el sistema.

- El frontend se enfoca en interacción, presentación, navegación, validación inicial y estados visuales.
- El backend se enfoca en reglas de negocio, persistencia, seguridad, validación autoritativa y comunicación con servicios externos.
- La base de datos se enfoca en consistencia, consulta, almacenamiento y disponibilidad de la información.
- Las integraciones externas se aíslan detrás de módulos propios para reducir acoplamiento y facilitar pruebas.

#### Señal de alarma

Cuando un cambio pequeño exige tocar tres capas a la vez de manera rutinaria, el sistema tiene un problema de acoplamiento. Cuando un archivo mezcla consultas a la base de datos con lógica de presentación, tiene un problema de cohesión. Ambas señales se detectan leyendo código, no esperando a que falle.

### 4. Características del desarrollo Full Stack

El desarrollo Full Stack es la práctica de construir y razonar sobre todas las capas de una aplicación web: la interfaz, los servicios, la persistencia y la operación. No nació como una moda laboral sino como una consecuencia técnica: cuando frontend y backend comparten lenguaje y se comunican por contratos claros,

una sola persona o un equipo reducido puede llevar una funcionalidad desde la pantalla hasta la base de datos. La llegada de Node.js, que llevó JavaScript al servidor, consolidó esta posibilidad al unificar el lenguaje de ambos extremos.

- **Visión transversal:** el desarrollador entiende el recorrido completo de un dato, desde el formulario hasta el documento persistido y de vuelta a la pantalla.
- **Contratos explícitos:** la comunicación entre capas se define mediante APIs documentadas, no mediante conocimiento implícito.
- **Lenguaje compartido:** en el ecosistema JavaScript, frontend y backend usan el mismo lenguaje, lo que reduce fricción cognitiva y permite compartir validaciones y tipos.
- **Iteración corta:** una misma persona puede prototipar una funcionalidad de extremo a extremo y validar la idea antes de invertir en especializaciones.
- **Responsabilidad por el resultado:** el éxito se mide en el flujo completo funcionando, no en una capa entregada de forma aislada.

## 5. Tipos de desarrollo Full Stack

No todo desarrollo Full Stack es igual. La industria distingue variantes según dónde se ejecuta la lógica de presentación y cómo se reparte el trabajo entre cliente y servidor. Conocer estas variantes permite leer ofertas tecnológicas con criterio: detrás de cada una hay una decisión sobre rendimiento percibido, posicionamiento en buscadores, complejidad operativa y experiencia del desarrollador. Este curso trabaja principalmente la variante SPA con API —una interfaz React que consume servicios construidos sobre Node.js—, por ser la que mejor expone los conceptos de contrato, estado y separación de capas, y la que con mayor frecuencia aparece en los equipos de desarrollo de la región.

| Tipo                                  | Descripción                                                                       | Cuándo conviene                                                              |
|---------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Renderizado en servidor (SSR clásico) | El servidor genera HTML completo en cada solicitud; el navegador muestra páginas. | Sitios con mucho contenido, indexación prioritaria y poca interacción.       |
| SPA con API                           | El cliente es una aplicación JavaScript que consume una API REST o GraphQL.       | Aplicaciones interactivas con sesiones largas y estado complejo en pantalla. |
| Híbrido / universal                   | Frameworks que combinan renderizado en servidor con hidratación en el cliente.    | Productos que necesitan a la vez carga inicial rápida e interactividad rica. |
| Sitios estáticos con servicios        | Páginas pregeneradas que delegan lo dinámico a APIs y funciones en la nube.       | Contenido mayormente estable con islas puntuales de funcionalidad.           |

## 6. Frameworks frontend

Un framework frontend resuelve un problema concreto: mantener sincronizada la pantalla con los datos. Hacerlo manualmente, manipulando el documento HTML instrucción por instrucción, se vuelve inmanejable en cuanto la interfaz tiene más de unos pocos estados. Los frameworks modernos proponen un modelo declarativo: el desarrollador describe cómo debe verse la interfaz para cada estado posible, y el framework calcula los cambios mínimos necesarios cuando el estado cambia.

| Framework | Modelo y filosofía                                                                                                   | Consideraciones                                                                                                      |
|-----------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| React     | Biblioteca centrada en componentes y funciones; la interfaz es una función del estado. Ecosistema enorme y flexible. | Exige decidir por cuenta propia enrutamiento, manejo de estado global y estructura; esa libertad es virtud y riesgo. |
| Angular   | Framework completo y opinado: enrutamiento, formularios, inyección de dependencias y pruebas vienen integrados.      | Curva de aprendizaje mayor; aporta uniformidad valiosa en equipos grandes y proyectos de larga vida.                 |
| Vue       | Punto intermedio: progresivo, con plantillas cercanas al HTML y reactividad integrada.                               | Adopción sólida aunque menor en el mercado laboral regional; muy apreciado por su claridad inicial.                  |

Los tres comparten las ideas estructurales que importan para este curso: componentes reutilizables, estado que dirige la presentación y un flujo de datos predecible. Quien domina esos conceptos en uno puede migrar a otro con esfuerzo razonable. El curso adopta React porque su modelo de componentes como funciones expone los conceptos con la menor cantidad de ceremonia, y porque su demanda en la industria amplía las oportunidades del estudiante; pero el criterio de selección —adecuación al problema, equipo y ecosistema— pesa más que el nombre elegido.

## 7. Frameworks backend

Del lado del servidor, un framework resuelve otro problema: recibir solicitudes HTTP, dirigirlas a la función correcta, validar entradas y producir respuestas, todo sin reescribir la infraestructura común en cada proyecto. Sobre Node.js conviven opciones con filosofías distintas, y compararlas enseña más que memorizar una.

| Framework | Modelo y filosofía                                                                                           | Consideraciones                                                                                                       |
|-----------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Express   | Minimalista y veterano: rutas, middlewares y poco más. El desarrollador compone el resto.                    | Estándar de facto para aprender los fundamentos de HTTP; su simplicidad obliga a entender cada pieza que se agrega.   |
| Hono      | Ligero y moderno, diseñado para múltiples entornos de ejecución (Node, Bun, borde de red) con tipado fuerte. | API similar a Express pero contemporánea; atractivo cuando el despliegue ocurre en plataformas serverless o de borde. |
| NestJS    | Framework estructurado con módulos, controladores e inyección de dependencias, inspirado en Angular.         | Impone arquitectura desde el primer día; valioso en sistemas grandes, excesivo para servicios pequeños.               |

El patrón subyacente es el mismo en los tres: una cadena de funciones —los middlewares— procesa cada solicitud antes de llegar al controlador que produce la respuesta. Autenticación, registro de actividad, validación y manejo de errores se insertan en esa cadena. El curso parte de Express porque deja ese mecanismo a la vista, y porque casi toda la documentación del ecosistema Node lo asume como referencia.

## 8. Integración frontend-backend

La integración entre capas es donde el diseño Full Stack se vuelve concreto. El punto de encuentro es el contrato de la API: el acuerdo sobre qué rutas existen, qué datos esperan, qué devuelven y qué errores pueden producir. Un contrato claro permite que frontend y backend se desarrollen en paralelo y se prueben

por separado; un contrato implícito, que vive solo en la memoria de quien lo escribió, garantiza fricciones en la integración.

El vehículo habitual del contrato es HTTP con JSON. El frontend emite solicitudes con un verbo que expresa la intención —GET para consultar, POST para crear, PUT para actualizar, DELETE para eliminar— y el backend responde con un código de estado que expresa el resultado y un cuerpo JSON con los datos o el error. Esta convención, conocida como estilo REST, no es la única posible, pero su simplicidad la mantiene como base del desarrollo profesional.

```
// El contrato visto desde el frontend
const respuesta = await fetch("/api/solicitudes", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ asunto: "Retiro de curso" }),
});
if (!respuesta.ok) {
  // 400: datos inválidos · 401: sin sesión · 500: falla interna
}
const solicitud = await respuesta.json();
```

- Documentar cada endpoint: ruta, verbo, cuerpo esperado, respuesta y errores posibles.
- Tratar los códigos de estado como parte del contrato: el frontend decide qué mostrar según el código recibido.
- Validar en ambos extremos con propósitos distintos: en el frontend por experiencia de usuario, en el backend por seguridad.
- Aislar las llamadas HTTP en módulos de servicio del frontend, de modo que los componentes no conozcan rutas ni formatos.

## 9. Stacks tecnológicos más utilizados

Un stack tecnológico es un conjunto de herramientas que trabajan juntas a lo largo de todas las capas. Los stacks con nombre propio —MERN, PERN, T3— son combinaciones que la industria validó por compatibilidad y comunidad, no recetas obligatorias. El programa del curso menciona Node.js, Express, React y MongoDB, la combinación conocida como MERN; lo importante no es memorizar siglas, sino entender qué decisión encarna cada componente y qué se gana o se pierde al sustituirlo.

| Stack | Componentes                                 | Trade-off principal                                                                                                                      |
|-------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| MERN  | MongoDB, Express, React, Node.js.           | Documentos flexibles que aceleran el prototipado; la falta de esquema rígido exige disciplina propia para mantener consistencia.         |
| PERN  | PostgreSQL, Express, React, Node.js.        | Relaciones e integridad garantizadas por la base de datos; el modelado inicial exige más diseño anticipado.                              |
| T3    | Next.js, TypeScript, tRPC/Prisma, Tailwind. | Tipado de extremo a extremo y contratos verificados por el compilador; más herramientas que aprender y mayor dependencia del ecosistema. |

### Cómo se elige un stack

La elección correcta responde a criterios, no a popularidad: adecuación al tipo de datos y de interacción, curva de aprendizaje asumible por el equipo en el tiempo disponible, madurez del ecosistema y la documentación, soporte para los requisitos de seguridad, y costo de operación. Un stack excelente para una empresa con cincuenta ingenieros puede ser una mala decisión para un equipo de tres estudiantes con un semestre por delante.

| Criterio               | Qué evaluar                                                               | Ejemplo de decisión                                                          |
|------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Adecuación al problema | Si la tecnología soporta el tipo de interacción, datos y flujo requerido. | React es útil para interfaces dinámicas con estados de usuario.              |
| Curva de aprendizaje   | Si el equipo puede construir con calidad dentro del tiempo disponible.    | Express permite iniciar APIs REST sin una arquitectura excesivamente pesada. |
| Ecosistema             | Disponibilidad de paquetes, documentación y comunidad.                    | npm facilita instalar middleware, validadores y herramientas de prueba.      |
| Persistencia           | Tipo de datos, volumen esperado y relaciones.                             | MongoDB se ajusta bien a documentos flexibles y prototipos iterativos.       |
| Seguridad              | Soporte para autenticación, protección de secretos y validación.          | JWT y bcrypt cubren una base mínima de control de acceso.                    |

## 10. El rol profesional del desarrollador Full Stack

El desarrollador Full Stack participa en varias etapas del proyecto: comprensión del problema, diseño de arquitectura, implementación de backend, modelado de datos, construcción de interfaz, pruebas, documentación y despliegue. En equipos reales estas responsabilidades pueden dividirse entre especialistas; el valor diferencial del perfil Full Stack está en entender las conexiones, traducir entre especialidades y detectar problemas que solo se ven mirando el sistema completo.

- Traducir requerimientos del negocio en decisiones de arquitectura y contratos de API.
- Implementar funcionalidades de extremo a extremo: modelo de datos, servicio, interfaz y prueba del flujo.
- Diagnosticar fallas cruzando capas: distinguir si un error nace en la pantalla, en la red, en el servicio o en los datos.
- Cuidar la seguridad como propiedad transversal: validación autoritativa, manejo de sesiones y protección de secretos.
- Documentar decisiones y contratos para que el sistema sea comprensible por otros, incluida la versión futura de uno mismo.
- Comunicarse con perfiles no técnicos explicando consecuencias de las decisiones, no detalles de implementación.

### Responsabilidad profesional

Un desarrollador Full Stack no debe esconder decisiones detrás de frases como “así lo hace el framework”. Debe explicar por qué una ruta existe, por qué un dato se valida, por qué una entidad se modela de cierta forma y cómo se prueba el flujo completo. La herramienta puede automatizar el trabajo; nunca la justificación.

Conviene también delimitar el mito del perfil omnisciente. Full Stack no significa dominar con la misma profundidad cada tecnología existente; significa sostener un mapa mental completo del sistema y la capacidad de profundizar en cualquier punto cuando el problema lo exige. La industria valora este perfil precisamente porque reduce los espacios donde los problemas se pierden “entre equipos”.

## 11. El ciclo de desarrollo de una aplicación Full Stack

Construir una aplicación completa exige un orden de trabajo. El ciclo que se presenta a continuación no es un trámite burocrático: cada fase existe porque omitirla tiene un costo conocido. Saltarse la delimitación del MVP produce proyectos que nunca terminan; saltarse los datos semilla produce interfaces imposibles de probar; saltarse la integración temprana acumula sorpresas para el final, cuando ya no hay tiempo de corregirlas.

- 1 Formular el problema y delimitar el MVP: qué hace la versión mínima y, sobre todo, qué deja explícitamente fuera.
- 2 Diseñar requerimientos y arquitectura: capas, contratos de API y modelo de datos antes de escribir código.
- 3 Configurar repositorio, entorno y convenciones: nombres, ramas, variables de entorno y estructura de carpetas.
- 4 Construir un backend mínimo con las rutas principales y datos semilla que permitan probar de inmediato.
- 5 Construir el frontend conectado a servicios reales, no a datos simulados que ocultan los problemas de integración.
- 6 Agregar seguridad, validaciones y manejo de errores sobre el flujo que ya funciona.
- 7 Probar la integración de extremo a extremo, corregir hallazgos y documentar el uso.
- 8 Preparar la entrega, la demostración y la reflexión técnica sobre lo aprendido.

Este ciclo se recorre de forma distinta según la metodología del equipo. Bajo un enfoque en cascada, cada fase se completa antes de iniciar la siguiente, lo que aporta orden pero descubre los errores de diseño demasiado tarde. Bajo enfoques ágiles —los que dominan el desarrollo Full Stack actual—, el ciclo completo se recorre en iteraciones cortas: cada sprint entrega una porción funcional que atraviesa todas las capas, y la retroalimentación de cada iteración corrige el rumbo de la siguiente. La arquitectura en capas y las metodologías iterativas se refuerzan mutuamente: los contratos claros entre capas son lo que permite repartir el trabajo de un sprint entre varias personas sin bloqueos.

| Fase del ciclo           | Responsabilidad principal del Full Stack                      | Riesgo si se omite                                       |
|--------------------------|---------------------------------------------------------------|----------------------------------------------------------|
| Formulación y alcance    | Negociar un MVP realista y registrar lo excluido.             | Proyecto sin final definido; esfuerzo disperso.          |
| Diseño de arquitectura   | Definir capas, contratos y modelo de datos.                   | Decisiones improvisadas que se petrifican en el código.  |
| Construcción iterativa   | Entregar flujos completos por iteración, integrando temprano. | Capas que funcionan aisladas pero no juntas.             |
| Pruebas y endurecimiento | Verificar flujos, errores y seguridad de extremo a extremo.   | Demostraciones que fallan; vulnerabilidades silenciosas. |

| Fase del ciclo      | Responsabilidad principal del Full Stack                 | Riesgo si se omite                               |
|---------------------|----------------------------------------------------------|--------------------------------------------------|
| Entrega y operación | Desplegar, documentar y dejar el sistema diagnosticable. | Software que solo corre en la máquina del autor. |

## 12. Buenas prácticas

Las buenas prácticas no son rituales: cada una previene una clase concreta de error. Separar responsabilidades evita que un cambio visual rompa una regla de negocio; las variables de entorno evitan que un secreto termine publicado en un repositorio; la validación en el backend evita confiar en un cliente que cualquier persona puede manipular. El estudiante debe poder enunciar, para cada práctica, el problema que previene.

- Separar responsabilidades: componentes de UI, servicios HTTP, controladores, modelos y utilidades en módulos distintos.
- Usar variables de entorno para configuración sensible; ningún secreto viaja en el código fuente.
- Nombrar rutas, entidades y componentes de forma consistente: la convención es documentación gratuita.
- Validar datos en el frontend por experiencia de usuario y en el backend por seguridad; solo la segunda es autoritativa.
- Documentar endpoints y decisiones relevantes en el README, en el momento en que se toman.
- Probar el flujo principal de extremo a extremo antes de agregar funcionalidades secundarias.
- Mantener commits pequeños y descriptivos que cuenten la historia del desarrollo.

## 13. Caso guía: seguimiento de solicitudes académicas

Considere una aplicación para gestionar solicitudes académicas. El estudiante crea una solicitud, el sistema la guarda, un administrador cambia su estado y el estudiante consulta el avance. Este flujo, deliberadamente simple, obliga a ejercitar casi todo lo discutido en el capítulo: autenticación, formularios, rutas REST, modelo de datos, estados del dominio, autorización por roles y pruebas del recorrido completo. Cada concepto teórico de las secciones anteriores tiene aquí una consecuencia visible.

| Decisión            | Aplicación en el caso                                                                           | Riesgo si se ignora                                                   |
|---------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Separación de capas | React consume una API Express; MongoDB no se expone al navegador.                               | Datos sensibles expuestos o lógica duplicada.                         |
| Contrato de la API  | Rutas documentadas para crear, listar y actualizar solicitudes, con códigos de error definidos. | Integración por ensayo y error; pantallas que fallan sin explicación. |
| Estado del dominio  | La solicitud tiene estados explícitos: creada, en revisión, aprobada, rechazada.                | Interfaz ambigua y reportes inconsistentes.                           |
| Autorización        | Solo el administrador cambia estados; el backend lo verifica en cada solicitud.                 | Usuarios podrían modificar datos no permitidos.                       |

| Decisión | Aplicación en el caso                                          | Riesgo si se ignora                                 |
|----------|----------------------------------------------------------------|-----------------------------------------------------|
| Pruebas  | Crear, listar y actualizar una solicitud de extremo a extremo. | El flujo puede fallar aunque las pantallas existan. |

Vale la pena recorrer mentalmente un dato a través del sistema: el asunto que el estudiante escribe en el formulario viaja como JSON en un POST, atraviesa los middlewares de autenticación y validación, se convierte en un documento persistido con su estado inicial, y regresa más tarde como parte de una lista que un componente React transforma en pantalla. Quien puede narrar ese recorrido sin saltos posee la competencia central de este capítulo.

## 14. Actividades sugeridas

- Dibujar la arquitectura de un proyecto propio con al menos cuatro capas, indicando qué conoce cada capa de la siguiente.
- Identificar tres decisiones técnicas del proyecto y justificar cada una con un requerimiento concreto.
- Describir por escrito un flujo completo desde una acción del usuario hasta la persistencia y de vuelta a la pantalla.
- Comparar dos stacks posibles para el mismo MVP y argumentar la elección usando los criterios de la sección 9.
- Clasificar una aplicación conocida (banca en línea, mensajería, comercio electrónico) según los tipos de la sección 5 y justificar la clasificación.
- Detectar en código propio o ajeno un caso de baja cohesión o alto acoplamiento y proponer la corrección.

## 15. Rúbrica breve

| Criterio         | Excelente                                                           | Satisfactorio                              | Insuficiente                             |
|------------------|---------------------------------------------------------------------|--------------------------------------------|------------------------------------------|
| Arquitectura     | Capas y responsabilidades claras, conectadas con requerimientos.    | Capas identificadas con algunas omisiones. | Lista tecnologías sin explicar relación. |
| Justificación    | Decisiones técnicas defendibles y alineadas al MVP.                 | Justificación parcial.                     | Decisiones arbitrarias.                  |
| Flujo            | Explica datos, rutas, validación y respuesta del usuario.           | Explica el flujo general.                  | No conecta capas.                        |
| Buenas prácticas | Incluye separación, variables, validación, documentación y pruebas. | Menciona algunas prácticas.                | No considera calidad.                    |

## 16. Cierre

La arquitectura Full Stack es, ante todo, una herramienta de pensamiento. La historia de las arquitecturas enseña que cada estilo responde a un problema; la teoría de capas enseña a contener el cambio; los frameworks y stacks materializan esas ideas con nombres concretos; y el ciclo de desarrollo las convierte en producto verificable. El estudiante que termina este capítulo debe poder responder, con argumentos y no con nombres de moda: por qué su aplicación separa capas, qué tipo de desarrollo Full Stack practica, qué encarna cada componente de su stack, qué hace en cada fase del ciclo y qué error previene cada buena

práctica que adopta. El siguiente capítulo desciende al terreno: el entorno, las herramientas y la configuración con las que estas ideas se vuelven código en ejecución.